# Kriging

## CMDA 4654 Project 2 - Group 1

Baker Dean, Toshaani Dewan, Nick Fitti, Nico Ottobre, Margaret Smith

April 26th, 2026

# Spatial / Geostatistical Data

**What is spatial data?**

- Observations collected at specific locations
- Each point has coordinates $(x, y)$ and a measured value $Z(x)$
- Location matters because nearby points often behave similarly
- Values may change across space rather than randomly

**Key Idea: Spatial Dependence**

- Points close together tend to have more similar values
- Points farther apart tend to be less related
- This relationship can be modeled statistically

$$\text{Nearby} \Rightarrow \text{Similar}$$

$$\text{Farther Away} \Rightarrow \text{Less Similar}$$

# Motivation for Kriging

**Why was Kriging developed?**

- Developed in the 1950s by South African mining engineer Danie Krige
- Created to estimate gold grades from limited borehole samples/estimate ore concentrations in unsampled regions
- Later formalized mathematically by Georges Matheron
- Designed to improve prediction accuracy over simpler methods

**Why do we need it?**

- Real-world data is often sparse, time-consuming or expensive to collect
- Measuring every location is impractical
- Need reliable estimates between sampled points
- Need uncertainty measures for decision-making

**What is the goal?**

- Predict unknown values using nearby observations
- Use spatial autocorrelation efficiently
- Reduce bias in predictions
- Minimize prediction error variance

# What is Kriging?

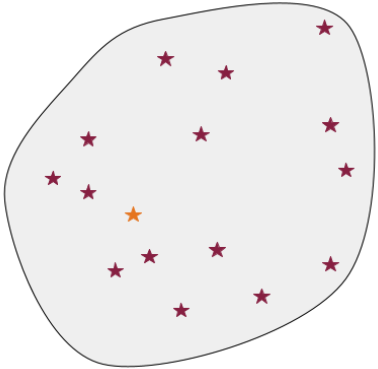
**Definition**

- Kriging is a geostatistical interpolation method
- Estimates values at unknown locations using nearby observed data
- Uses both distance and spatial correlation between points
- Provides prediction uncertainty

- An observation will be more similar to data that is "close" in space compared to those "far" in space.

"Closeness" will be determined by the spatial correlation between points in the given field of data.

The "Data Field"



# Kriging - Overview

**Estimator:**

$$Z^*(x_0) = \sum_{i=1}^{n} \lambda_i Z(x_i) = \lambda^T \mathbf{z}$$

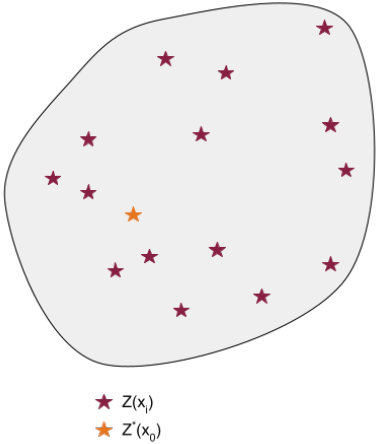
- $Z(x_i) =$ known observation values at location $x_i$
- $\lambda_i =$ optimal weights
- $Z^*(x_0) =$ predicted value at new location $x_0$
- $n =$ number of locations where data is observed

**Core Idea:**

- Closer points receive more weight
- Farther points receive less weight

Near $\Rightarrow$ More Influence

Far $\Rightarrow$ Less Influence



# Motivation for Kriging (cont.)

**When is Kriging a Good Choice?**

- Data has geographic or location-based patterns
- Nearby points are likely related
- Only limited observations are available
- Accurate interpolation is needed between sampled locations
- Prediction uncertainty is important for decisions

$$\text{Spatial Data} + \text{Limited Samples} \Rightarrow \text{Kriging}$$

# Common Applications of Kriging

**Environmental & Earth Sciences**

- Rainfall and temperature mapping
- Air pollution estimation
- Soil quality / agriculture mapping
- Groundwater and hydrogeology
- Oil, gas, and mineral exploration

**Modern Analytics**

- Public health risk mapping
- Urban planning
- Astronomy data interpolation

**Engineering & Industry**

- Real estate valuation
- Circuit design optimization
- Resource planning

**Remote Sensing**

- Filling missing spatial data
- Surface prediction from satellite data

**Sports Analytics**

- Shot efficiency by location
- Court / field heat maps
- Player movement zones

# Variations of Kriging

*Several Kriging methods exist depending on assumptions about the mean and spatial trend.*

**Most Common Types**

- **Simple Kriging** Known constant mean
- **Ordinary Kriging** Unknown but constant mean
- **Universal Kriging** Allows a spatial trend in the mean - Mean follows a trend across space or data

**Other Extensions**

- Block Kriging
- Co-Kriging
- Indicator Kriging
- Bayesian Kriging
- Poisson Kriging

Our Focus: **Ordinary Kriging**

# Advantages of Kriging

**Prediction Strength**

- Produces accurate estimates at unsampled locations
- Uses spatial correlation, not only distance
- Weights are chosen to minimize estimation variance
- Often outperforms simple inverse distance weighting

**Built-in Uncertainty**

- Predicts a value *and* prediction error
- Helps measure confidence in estimates
- Useful when making real decisions from data
- Identifies areas where more samples may be needed

**Flexible Across Fields**

- Environment, health, geology, engineering, sports analytics

# The Kriging Model

<u>Generic Model:</u>

$$Z(x) = \mu(x) + \epsilon(x)$$

$\epsilon(x)$ is assumed to have a zero mean and a corresponding variogram model with function $C(x_i, x_j)$. This is also notated as $C(\mathbf{h})$, where $\mathbf{h}$ is the difference vector between two locations $x_i$ and $x_j$.

Note: This assumes that the variation in the data is a function of the distance between points.

As previously mentioned, we will be focusing on *Ordinary Kriging*.

Ordinary Kriging assumes an unknown stationary mean. That is, the mean of the response variable is independent of location. This simplifies the above model to:

$$Z(x) = \mu + \epsilon(x)$$

Additionally, we assume isotropy; any pair of locations that are the same distance from each other, regardless of direction, have the same spatial correlation.

# Solving for the Kriging Weights

To solve for the weights $\lambda_i$ in

$$Z^*(x_0) = \sum_{i=1}^{n} \lambda_i Z(x_i),$$

To estimate the weights $\lambda_i$ is equivalent to finding the $\lambda_i$ that minimizes the error variance

$$\text{min } \sigma_e^2 = \text{min Var}\left[Z(x) - Z^*(x)\right]$$

such that the estimator is unbiased, i.e. $\text{E}\left[Z(x) - Z^*(x)\right] = 0$. (This is satisfied by assuming a stationary mean.)
So, the weights are constrained to satisfy

$$\sum_{i=1}^{n} \lambda_i = 1 \quad \text{or} \quad \lambda^T \mathbf{1} = 1$$

and we are able to express the minimization problem as,

$$\text{min } \sigma_e^2 = \text{min } E\left[(Z(x) - Z^*(x))^2\right]$$

To solve this constrained optimization problem, the method of Lagrange multiplier is used.

# Solving for the Kriging Weights (cont'd)

Not all proof steps are shown, but a thorough reference here provides a clear explanation.

After some algebraic manipulation, we arrive at minimizing

$$
\begin{aligned}
&C(0) - 2\sum_{i=1}^{n} \lambda_i C(x_0, x_i) + \sum_{i=1}^{n}\sum_{j=1}^{n} \lambda_i \lambda_j C(x_i, x_j) - 2\Lambda \left( \sum_{i=1}^{n} \lambda_i - 1 \right) \\
&= \mathbf{c_0} - 2\lambda^T \mathbf{c_{x_0}} + \lambda^T \mathbf{C} \lambda - 2\Lambda \left( \lambda^T \mathbf{1} - \mathbf{1} \right)
\end{aligned}
$$

by taking the partial derivatives with respect to the $\lambda_i$'s (or vector $\lambda$) and then the Lagrange multiplier $\Lambda$.

The solution results in solving the system

$$
\begin{bmatrix}
C(x_1, x_1) & C(x_1, x_2) & C(x_1, x_3) & \cdots & C(x_1, x_n) & 1 \\
C(x_2, x_1) & C(x_2, x_2) & C(x_2, x_3) & \cdots & C(x_2, x_n) & 1 \\
C(x_3, x_1) & C(x_3, x_2) & C(x_3, x_3) & \cdots & C(x_3, x_n) & 1 \\
\vdots & \vdots & & \ddots & & 1 \\
C(x_n, x_1) & C(x_n, x_2) & C(x_n, x_3) & \cdots & C(x_n, x_n) & 1 \\
1 & 1 & 1 & \cdots & 1 & 0
\end{bmatrix}
\begin{bmatrix}
\lambda_1 \\ \lambda_2 \\ \lambda_3 \\ \vdots \\ \lambda_n \\ \Lambda
\end{bmatrix}
=
\begin{bmatrix}
C(x_0, x_1) \\ C(x_0, x_2) \\ C(x_0, x_3) \\ \vdots \\ C(x_0, x_n) \\ 1
\end{bmatrix}
$$

# Solving for the Kriging Weights (cont'd)

The system given on the previous slide can be equivalently expressed as

$$\begin{bmatrix} \mathbf{C} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \lambda \\ \Lambda \end{bmatrix} = \begin{bmatrix} \mathbf{c}_0 \\ 0 \end{bmatrix}$$

And solved by simply inverting the matrix on the left hand side.

This matrix will be *in theory* invertible. The function $C(x_i, x_j)$ arises from variogram modeling, which uses conditionally negative definite functions. Mathematical depth aside, this class of functions will produce a positive definite matrix (which is invertible).
Note: As $n$ becomes large, the dimensions of the matrix could cause it to become ill-conditioned.

After solving for the kriging weights, we can find the error variance of our estimate $Z^*(x_0)$ as

$$\sigma_e^2 = C(0) - \sum_{i=1}^{n} \lambda_i C(x_0, x_i) + \Lambda$$

Last, due to construction, these weights will result in $Z^*(x_0)$ being a best linear unbiased estimator (BLUE).

# Variogram Model

We previously assumed that the spatial covariance can be modeled with a variogram function $2\gamma(\mathbf{h})$, or equivalently a semivariogram function $\gamma(\mathbf{h})$. (Though in spatial statistics these two terms are sometimes used interchangeably.)
In either case, we seek a function that describes the spatial covariance for the distance between two points.
This is achieved by first obtaining a *sample variogram* that is described by

$$2\widehat{\gamma}(\mathbf{h}) = \frac{1}{|N_{\mathbf{h}}|} \sum_{N_{\mathbf{h}}} (Z(x_i) - Z(x_j))^2$$

where $|N_{\mathbf{h}}|$ is the number of unique pairs in the set defined as

$$N_{\mathbf{h}} := \{(x_i, x_j) : |x_i - x_j| = h\} \text{ for } i, j = 1, \ldots, n$$

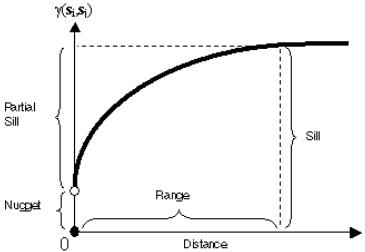
That is, we obtain $k$ discrete set of points with distances $h$ and corresponding average of variances $2\widehat{\gamma}(\mathbf{h})$.
Note: This can also be framed with $\gamma(\mathbf{h})$ denoting the semivariance occurring at distance $\mathbf{h}$.

# Variogram Model (cont'd)

Last, a variogram model is fit to the sample variogram. The optimal variogram will minimize the sum of squared error given by

$$\sum_{i=1}^{k} \left(2\widehat{\gamma}(\mathbf{h}) - 2\gamma(\mathbf{h}, \alpha)\right)^2$$

Where $\alpha$ is used to denote parameters of the variogram model.
Frequently, these parameters are the "nugget", "partial sill" and "range" in geospatial statistic terms.
Equivalently, these dictate the y-intercept, the difference in the maximum covariance and the y-intercept and the distance between the origin and where the maximum covariance is reached.
Image found [4]



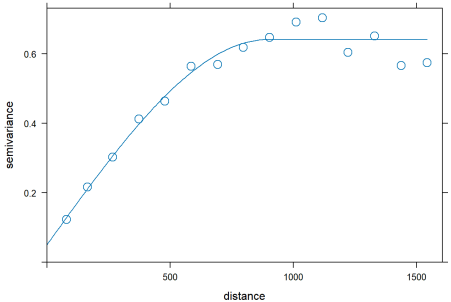
# Variogram Conclusion

A frequently used model is the spherical semivariance defined by

$$\gamma(\mathbf{h}) = \begin{cases} 0 & \text{if } h = 0 \\ \text{nugget } + (\text{partial sill}) \left(1 + \frac{3}{2}\left(\frac{h}{\text{range}}\right) + \frac{1}{2}\left(\frac{h}{\text{range}}\right)^3\right) & \text{if } 0 < h \leq \text{range} \\ \text{nugget } + \text{ partial sill} & \text{if } h > \text{range} \end{cases}$$

An example plot of the sample variogram points with a spherical variogram fit can look like the graph on the right.

Once a model has been fit, this function can be used to determine the values of $C(x_i, x_j)$ in the matrix $\begin{bmatrix} \mathbf{C} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix}$ and that of $C(x_0, x_i)$ in the vector $\begin{bmatrix} \mathbf{c}_0 \\ 0 \end{bmatrix}$ to solve for the weights.




From reference [11]

# Basic Example

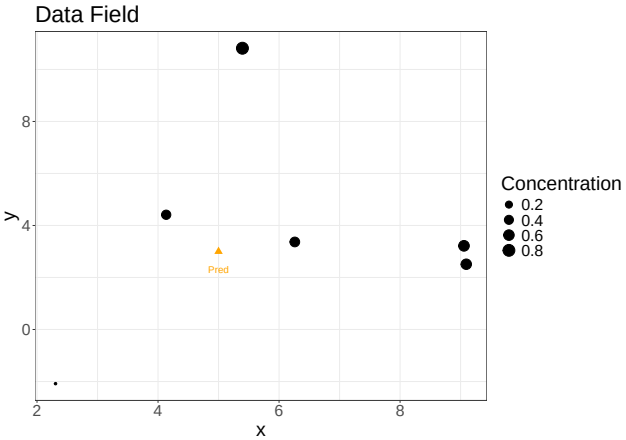
We generate a small data set to demonstrate how to compute the Kriging weights to predict data at a new location. A CSV for the generated dataset can be found here.

The $x$ and $y$ variables represent testing locations within a lake, and the $z$ variable is the concentration of algae at each testing location.
The `pred` value is a coordinate without a testing area that we want to estimate algae concentration at.

```
set.seed(1010)
# generating data at "known" locations
dat <- data.frame(x = rnorm(6, mean=5, sd=3),
                  y = rnorm(6, mean=5, sd=3))
# making there be a relationship between location and concentration
z <- (dat$x + dat$y)/(max(dat$x) + max(dat$y))
dat <- cbind(dat, z)

# location of prediction
pred = data.frame(x = 5, y = 3)
```

# Plotting the data field

```
ggplot(data = dat, aes(x=x, y=y, size=z)) + geom_point() + labs(size = "Concentration", title = "Data Field") +
  theme_bw() + annotate("point", x=5, y=3, color = "orange", size=3, shape = 17) +
  annotate("text", x = 5, y = 2.3, label = "Pred", color = "orange") + theme(text = element_text(size=22))
```



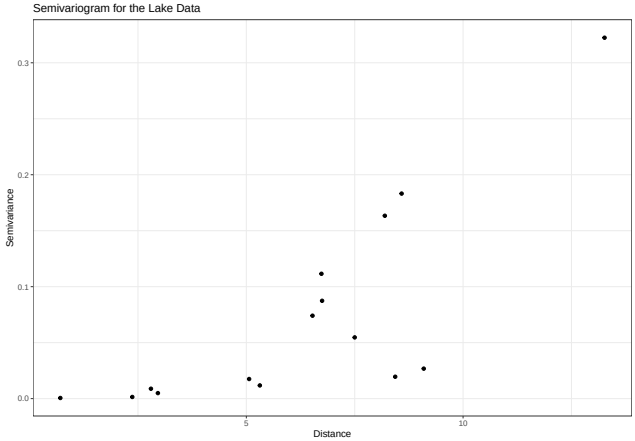
# Semivariance and Distance

Here, we calculate the distance between each unique set of points and the semivariance for each set of points.

```
# initializing df for semivariogram
semivariogram = data.frame(matrix(ncol = 2, nrow = 0))
colnames(semivariogram) = c('Distance', 'Semivariance')
# iterating over each location
for(i in 1:nrow(dat)){
  # at each location, we calculate the distance to all other points and corresponding
  # semivariance
  for(j in 1:nrow(dat)){
    if(j > i){
      dist = sqrt((dat$x[i] - dat$x[j])^2 + (dat$y[i] - dat$y[j])^2) # using L2 distance
      semivariance = 0.5*((dat$z[i] - dat$z[j])^2)
      semivariogram = rbind(semivariogram, data.frame("Distance" = dist,
                                                      "Semivariance" = semivariance))
    }
  }
}
```

# Semivariogram Plot

```
ggplot(semivariogram, aes(x = Distance, y = Semivariance)) + geom_point() +
  ggtitle("Semivariogram for the Lake Data") + theme(text = element_text(size = 22)) +
  theme_bw()
```



# Semivariogram Model Selection

There are a few different widely used semivariogram models. The most simple model is a linear model, and for this model there is no point in the graph where the semivariance flattens out and stops increasing, so the sill and range must be arbitrarily selected. As previously mentioned, a spherical model appears as a quadratic function that flattens out as the distance increases. Here, we choose the exponential model. It is similar to the spherical model but does not flatten out as completely.

We determine the nugget, sill, and range for the model. There is no y-intercept in the semivariogram plot, so we can assume that the model intercepts the y-axis at the origin.

```
nugget = 0
partial_sill = max(semivariogram$Semivariance) - nugget
range = semivariogram$Distance[10]
```

Here, we fit an exponential model for the sample semivariogram of the data, and this will be applied to a distance matrix of the points.

```
dist_mat = as.matrix(dist(dat[, 1:2]))
semivariogram2 = nugget + partial_sill*exp(-dist_mat/range)
```

# Set up Matrices and Vectors

We then calculate the vector for the Euclidean distance between the points in the plane and the new testing coordinates.

```
dist_pred = sqrt((dat$x - pred$x)^2 + (dat$y - pred$y)^2)
```

Next, we set up the matrices by adding a column and row of ones with a 0 in the bottom right spot in the matrix. This allows us to solve for the $\lambda_i$ and Lagrange multiplier values.

```
semi_mat = matrix(1, 7, 7)
semi_mat[1:6, 1:6] = semivariogram2
semi_mat[7, 7] = 0
```

Last, we calculate the semivariogram for the vector of the distances between the points and for the new prediction location.

```
semi_dist = nugget + partial_sill*exp(-dist_pred/range)
dist_pred2 = c(semi_dist, 1)
```

# Solving for Weights

We can solve for the weights and the Lagrange multiplier, $\Lambda$, by taking the inverse of the matrix and multiplying it by the vector for the distances between the points and the new prediction coordinates.

```
weights_v = solve(semi_mat) %*% dist_pred2
```

The weights are the first $n$ values of the resulting vector, where $n$ is the number of rows in the original data frame. The Lagrange multiplier is the final value in the vector, and this is used to calculate the variance of the prediction.

```
# extracting the values to be used separately
weights = weights_v[1:6]
Lagrange = weights_v[7]
```

# Prediction and Error

The predicted value is the sum of each weight multiplied by the corresponding $z$ value for the coordinates from the original data frame.

```
z_pred <- sum(weights*dat$z); z_pred
```

```
[1] 0.435394
```

The absolute error can be found by computing the "actual" concentration value from the formula we used to compute the data field as if the location was a part of the original dataset

```
z_true <- (pred$x + pred$y)/(max(dat$x) + max(dat$y)); z_true
```

```
[1] 0.401811
```

```
abs_err <- (z_pred - z_true); abs_err
```

```
[1] 0.033583
```

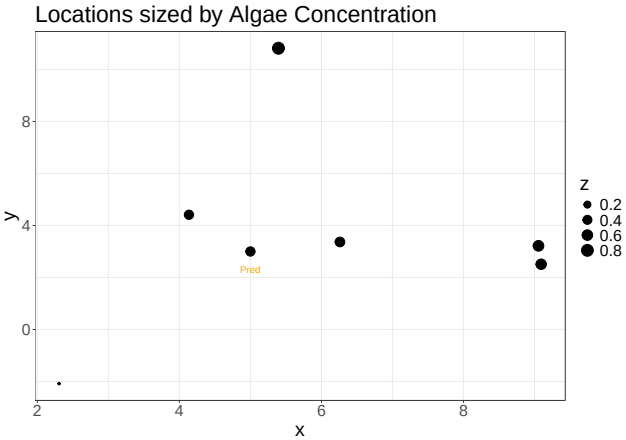
The variance of the errors was previously given as $\sigma_e^2 = C(0) + \sum_{i=1}^{n} \lambda_i C(x_0, x_i) + \Lambda$ and can be compute as follows. ($C(0)$ is the nugget, which is zero in this example.)

```
variance <- t(weights) %*% semi_dist + Lagrange; variance[1]
```

```
[1] 0.1578453
```

# Plot of Prediction

```
ggplot(dat2, aes(x = x, y = y, size = z)) +
  geom_point() + ggtitle("Locations sized by Algae Concentration") + theme_bw() +
  theme(text = element_text(size = 22)) + annotate("text", x = 5, y = 2.3, label = "Pred", color = "orange")
```

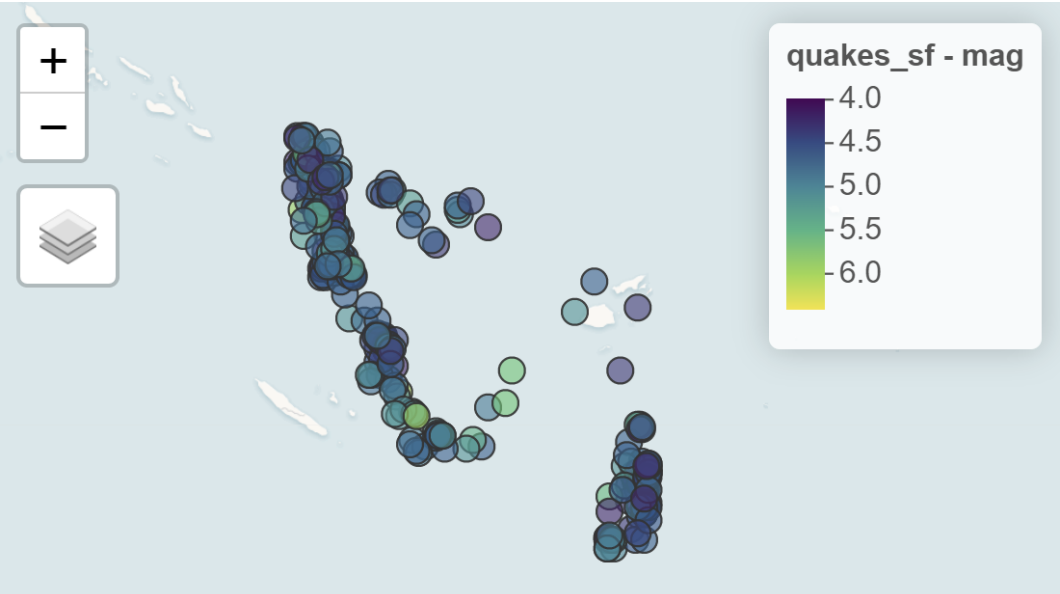


# Quakes - Introduction

We are going to look at an example using the quakes dataset which is built into R. This dataset has 1000 observations of earthquake data around the Fiji area and includes things like the earthquake's location via latitude and longitude, the depth of the earthquake in km, the earthquake's magnitude, and how many seismic stations detected the earthquake. This example is going to look into what kind of insights and predictions can be made on the magnitude of earthquakes through the use of kriging. A CSV for the quakes dataset can be found here.

```r
library(sf)
library(gstat)
library(mapview)
library(stars)

data(quakes)

# Convert data into a spatial object
quakes_sf <- st_as_sf(quakes, coords = c("long", "lat"), crs = 4326) %>%
  st_transform(32760)

# Plot on map
mapview(quakes_sf,
        zcol = "mag",
        map.types = "CartoDB.Voyager",
        legend = TRUE)
```

# Quakes - Plotted data

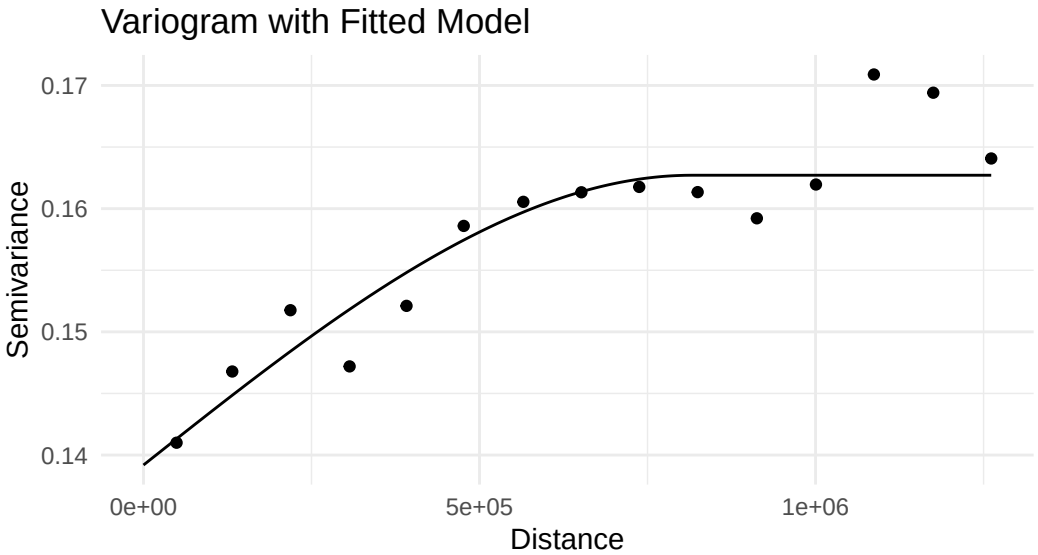


# Quakes - Variogram

The Variogram is one of the most important and insightful parts of kriging. The variogram is first estimated from the data, and then we choose a theoretical model to be fitted to it. The sample variogram helps guide you to choosing the best fit for your data. Choosing the best possible model that accurately represents the spatial structure of your data is very important, as it directly impacts the quality of your predictions.

```r
# Variogram
v <- variogram(mag ~ 1, quakes_sf)
# Selected Model
v_fit <- fit.variogram(v, vgm(psill = 0.2, model = "Sph", range = 150000, nugget = 0.05))
# Plotting
v_df <- as.data.frame(v)
v_line <- variogramLine(v_fit, maxdist = max(v_df$dist))
v_line_df <- as.data.frame(v_line)

ggplot() +
  geom_point(data = v_df, aes(x = dist, y = gamma)) +
  geom_line(data = v_line_df, aes(x = dist, y = gamma)) +
  labs(
    title = "Variogram with Fitted Model",
    x = "Distance",
    y = "Semivariance"
  ) +
  theme_minimal()
```

# Quakes - Variogram



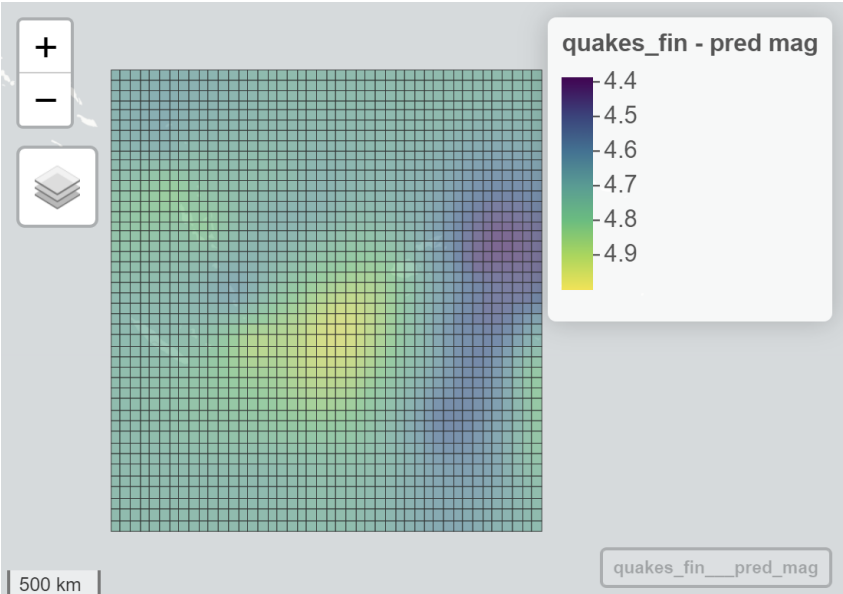
# Quakes - Kriging

Now that we have our model, we need some new points that we want to predict the magnitude for using kriging. To do this, we are going to create a grid of points and project it onto the Fiji area. This will allow us to use the points we have data for to make predictions on every point in the grid to see what their potential magnitude could look like.

Putting all this together, we can use the krige function from the gstat library to make predictions on the grid we just made and plot it onto a map.

```r
# create predictions grid in the Fiji area
pred_grid <- st_bbox(c(xmin = 163, ymin = -30, xmax = 185, ymax = -8), crs = 4326)
quakes_grid <- st_as_stars(pred_grid, dx = 0.5, dy = 0.5) %>%
  st_as_sf() %>%
  st_transform(32760)

# Kriging
quakes_kriged <- krige(mag ~ 1, quakes_sf, quakes_grid, model = v_fit)
quakes_kriged_map <- st_transform(quakes_kriged, 4326)
quakes_fin <- st_shift_longitude(quakes_kriged_map)
names(quakes_fin)[3] <- "pred mag"

# Plot on map
mapview(quakes_kriged,
        zcol = "pred mag")
```

# Quakes - Predictions



# Quakes - Limitations and Conclusion

**Limitations**

- Large areas with few data points tend to get pulled towards the global mean
- Extreme values have been "weakened" and pulled closer to the global mean
- Earthquakes have high variability when it comes to their magnitude

**Conclusion**

Our results give a decent idea of the general magnitude of earthquakes in areas we have data points for, but is not great for predicting the exact magnitude and it also struggles with predicting the magnitude in areas that lack data points. This highlights the idea that kriging is suited more towards interpolation rather than extrapolation. Kriging prefers smaller and more tight data, so trying to make predictions over a large area is going to result in over smoothing. This could be improved in this example by taking a smaller grid in a specific area of interest or by making the grid a more unique shape that follows the shape of the data rather than trying to get estimates for areas where we have no data points.

# Data Introduction

Points per shot is one of the biggest current topics of discussion in basketball analytics. It is calculated using two factors: a team or players chance of making a shot from a position on the floor and the point of value of a make from that position. Performing analysis on this topic requires gathering both of those values which can be done using the `hoopR` package.

```r
library(hoopR)

pbp <- load_nba_pbp(seasons = 2026)
player_box <- load_nba_player_box(seasons = 2026)

player_names <- player_box %>%
  distinct(athlete_id, athlete_display_name)

shots <- pbp %>%
  filter(shooting_play == TRUE,
         !is.na(coordinate_x), !is.na(coordinate_y), !grepl("Free Throw", type_text),
         season_type == 2) %>%
  select(athlete_id_1, coordinate_x, coordinate_y, score_value) %>%
  left_join(player_names, by = c("athlete_id_1" = "athlete_id")) %>%
  mutate(
    x = ifelse(coordinate_x > 0,
               -(coordinate_x - 46.75),
               coordinate_x + 46.75),
    y = ifelse(coordinate_x > 0, -coordinate_y, coordinate_y)) %>%
  filter(x <= 45) %>%
  select(athlete_display_name, score_value, x, y)
```

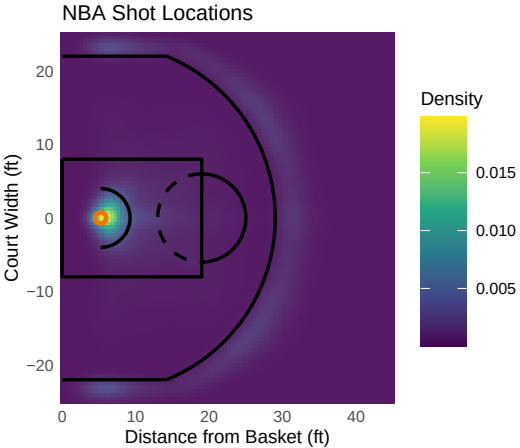
# Data Example

The data has been loaded and saved as csv for ease of access: Click here to access shots.csv.

Preview of the data

| athlete_display_name | team_name | score_value | x | y |
|---|---|---|---|---|
| Luka Garza | Boston Celtics | 3 | 28 | 12 |
| Franz Wagner | Orlando Magic | 3 | 30 | 13 |
| Luka Garza | Boston Celtics | 0 | 24 | 17 |
| Paolo Banchero | Orlando Magic | 2 | 7 | 1 |
| Baylor Scheierman | Boston Celtics | 3 | 38 | -1 |
| Jalen Suggs | Orlando Magic | 0 | 17 | 23 |
| Ron Harper Jr. | Boston Celtics | 0 | 27 | 15 |
| Franz Wagner | Orlando Magic | 3 | 4 | -23 |
| Baylor Scheierman | Boston Celtics | 0 | 31 | -8 |
| Wendell Carter Jr. | Orlando Magic | 0 | 15 | 23 |
| Baylor Scheierman | Boston Celtics | 0 | 8 | 23 |
| Paolo Banchero | Orlando Magic | 0 | 6 | -3 |
| Paolo Banchero | Orlando Magic | 0 | 8 | 2 |
| Paolo Banchero | Orlando Magic | 0 | 5 | 0 |
| Max Shulga | Boston Celtics | 0 | 15 | -2 |
| Jordan Walsh | Boston Celtics | 0 | 23 | -21 |
| Baylor Scheierman | Boston Celtics | 0 | 10 | -2 |
| Ron Harper Jr. | Boston Celtics | 2 | 8 | -4 |
| Ron Harper Jr. | Boston Celtics | 2 | 7 | -4 |
| Desmond Bane | Orlando Magic | 0 | 34 | 12 |

NBA 2025-26 Regular Season Heatmap



# Kriging Motivation

It is clear from the shot density layout on the previous slide that there are two main areas that teams take shots from:

- the three point line
- the rim

This makes sense given that shots at the rim are likely to be high percentage and shots from three point range are worth more points. The middle area between these two hot zones is fairly empty, and the league has been trending to make midrange jumpshots even less a part of offenses in the Modern Era. All of this evolution is in the pursuit of offensive efficiency which is where Kriging comes in.

Since the shot map is made up of x and y data, it can be treated similarly to a geostatistics field from more typical Kriging - it has high density areas and low gaps with a large number of very individualized spots. Kriging can be used to smooth over these less commonly attempted areas and give a picture of not only where teams shoot but how well they perform from these areas.

Combining those two areas will mean that Kriging can make a points per shot estimation from every location on the floor which can then be used to check if teams are running their offenses to peak efficiency.

# Kriging Execution

There will be three applications of Kriging used in this investigation but they will all generally follow the same flow. This code performs an example for attempt frequency:

- Make a grid of bins across the court containing true shot data

```
pred_grid <- make_pred_grid()
bins <- make_bins(team)
attempt_bins <- bins %>% select(x_bin, y_bin, attempts) %>% as.data.frame()
```

- Run the ‘autoKrige‘ function on the bins

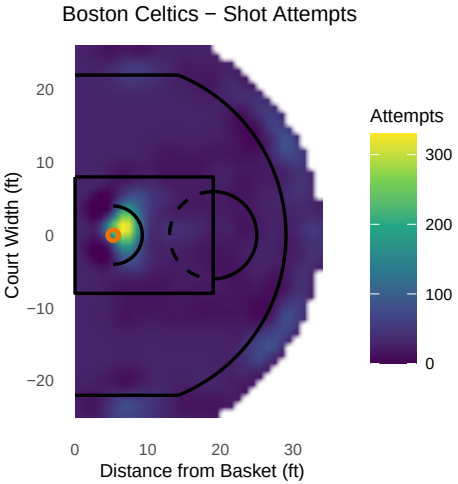
```
krig_attempts <- autoKrige(attempts ~ 1, input_data = attempt_bins, new_data = pred_grid)
```

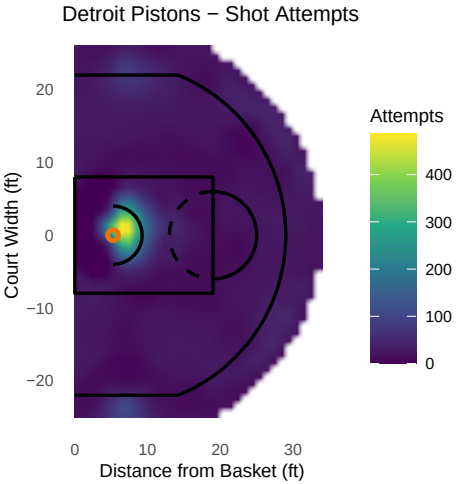
- Plot the output onto a prediction grid

```
ggplot(krig_attempts_df, aes(x = x, y = y, fill = attempts_pred)) +
    geom_raster(interpolate = TRUE) +
    draw_court() +
    scale_fill_viridis_c() +
    coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
    labs(title = paste(team, "- Shot Attempts"),
         x = "Distance from Basket (ft)",
         y = "Court Width (ft)",
         fill = "Attempts") +
    theme_minimal(base_size = 7) +
    theme(panel.grid = element_blank(), plot.margin = margin(t = 2, r = 0, b = 0, l = -3))
```

# Attempt Frequency Kriging

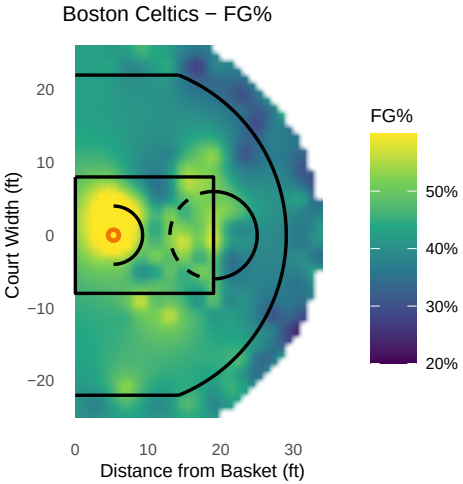
A typical picture of attempt frequency for a basketball team would consist of colored boxes or hexes that represent volume from each area of the court, but a custom Kriging can smooth this out into a continuous map using two foot by two foot bins.

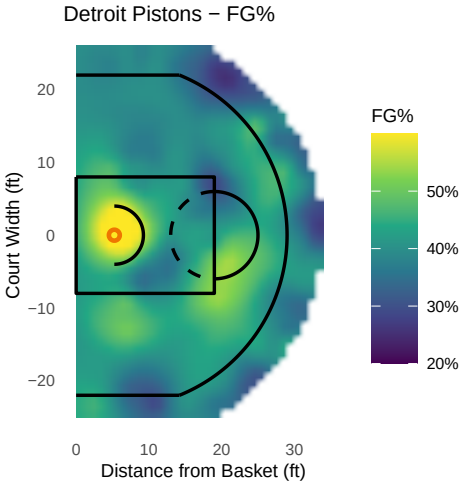


# FG% Kriging

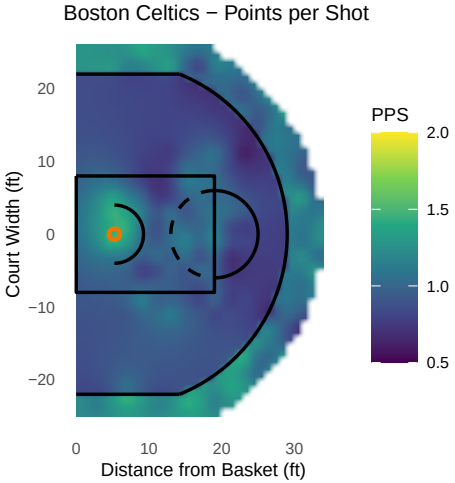
While the Kriging for shot attempts is somewhat trivial, it is extremely useful for field goal percentage. This statistic leads to a map that has continuous values across all the bins and the maps will differ more greatly team-to-team.

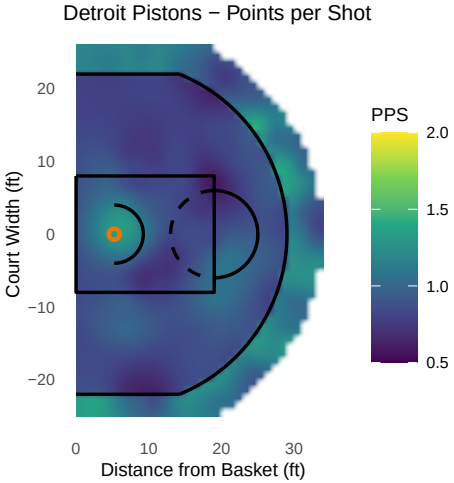


# Points Per Shot

As mentioned previously, points per shot is a metric that is used to determine the quality of a type of shot attempt. Now that the field goal percentage Kriging has supplied a likelihood of making a shot at any location on the floor, it is easy to take it a step further into point values of the shots.



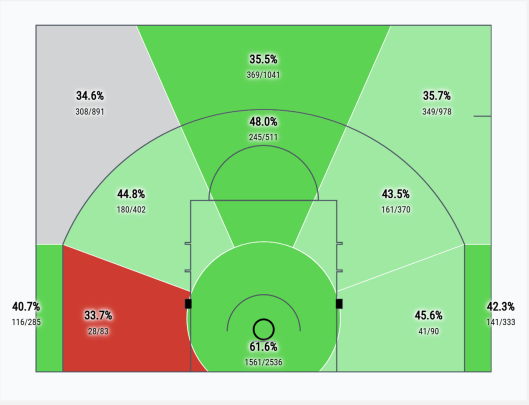



# Non-Kriging Approach

So the question becomes "How do these smooth maps help NBA teams improve their offenses?"

One of the main advantages that these plots provide is they make it much easier to get the full picture of shots across the floor. As a comparison, here would be a more standard look at a teams percentages and shot volumes from Synergy Sports.

The plot makes it hard to tell whether there's smaller deviations within the large group and doesn't calculate the point values of the shots (i.e. shooting 50% from 2 is the same as 33% from 3). Thus it is generally not very insightful about which shots are the most efficient, only offering general statistics about which areas the team has shot the most accurately from.



# Kriging Comparison

A smooth chart from Kriging much better captures the specific peaks and valleys of the shooting percentage and factoring in the point values gives real, actionable insights. Combining this with the shots attempted map can make it easy for a coach without a statistical background to see mismatches in shot value and frequency without complex interpretation.

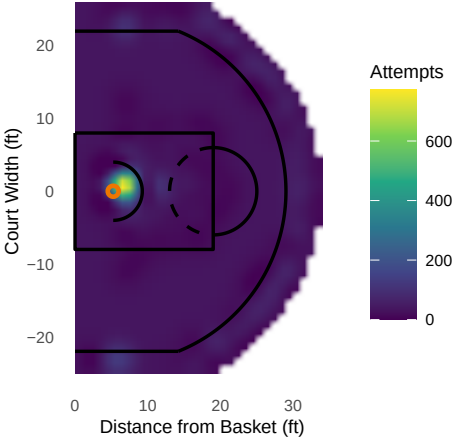
Running through every NBA team can make some interesting comparisons:

```
plot_team_impact <- function(team_name) {
   fg_obj <- plot_fg_pct(team_name)
   attempts_plot <- plot_attempts(team_name)
   pps_plot <- plot_pps(fg_obj$data)
   attempts_plot | pps_plot
}

nba_teams <- sort(unique(team_shots$team_name))

for (tm in nba_teams) {
   print(plot_team_impact(tm))
}
```

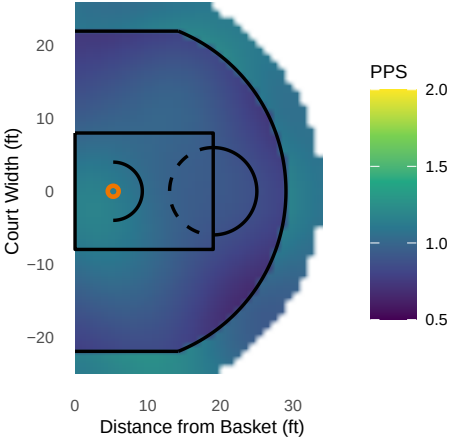
# Team Takeaways

Some teams could gain insights by looking at their charts side-by-side, like the Cleveland Cavaliers

Cleveland Cavaliers – Shot Attempts

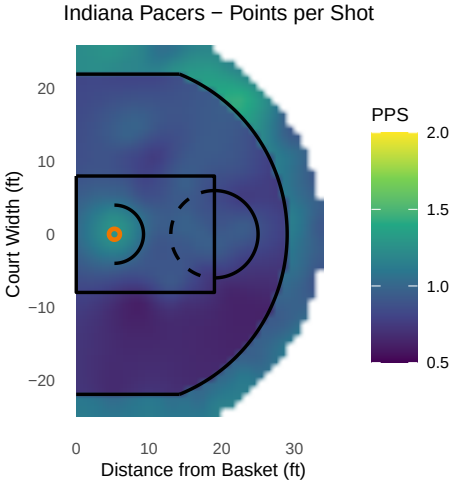


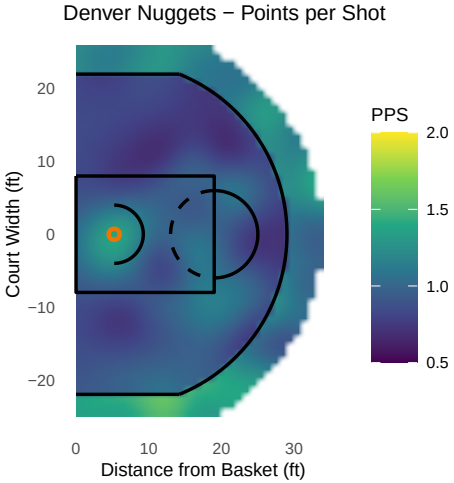

Cleveland Cavaliers – Points per Shot



# Points per Shot Patterns

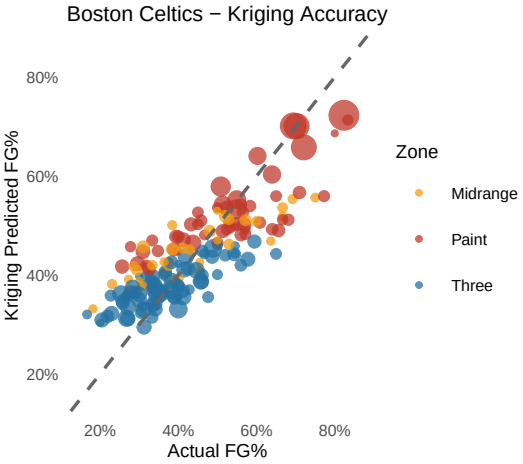
Some teams also have clear patterns that favor shooting from one side of the floor

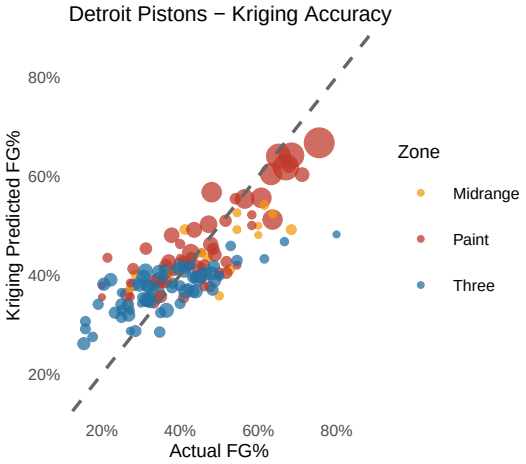


# Accuracy of Predictions

Of course, the predicted maps are only useful if they are accurate. So how well did Kriging do with smoothing the shooting data?

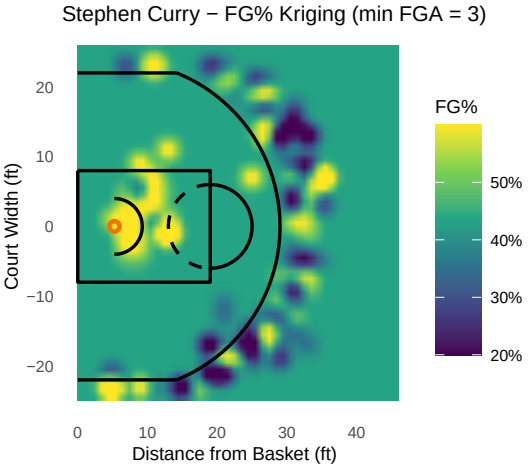


# Kriging Limitations

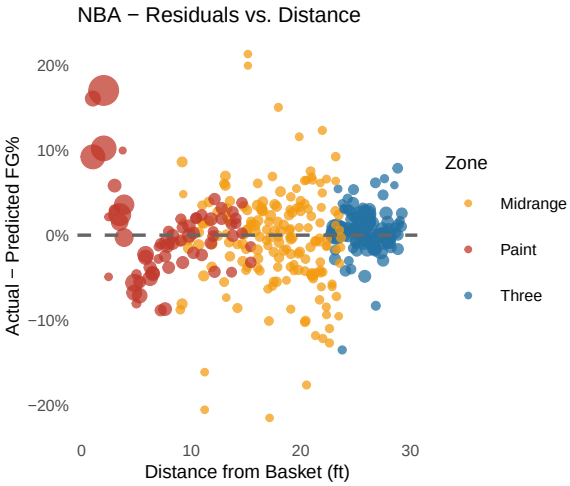
Kriging has 2 major limitations: sparse data and giant data. The giant data problem is avoided for this case by filtering out all of the free throws and plays that did not result in a shot attempt from the large play-by-play dataset, but slight manual adjustments had to be made for sparse data:

This plot of Stephen Curry's individual FG% Kriging from last season makes it clear that having too little data can be a problem as well. Not only are the bins that have enough shots to analyze somewhat blocky, but the areas where there are no data have been pulled towards the global mean which is obviously unreasonable given the context.
This emphasizes that Kriging is far better at interpolation than extrapolation.



# Conclusions

A plot of the residuals from the predictions against the distance from the basket of the shots gives some insights about how Kriging outputs tend to work - the model underpredicted the almost outlier-like high percentages on dunks and layups near the basket, was most wild for the midrange where the data is far more sparse and inconsistent, and then was fairly tight and accurate for three point shots. This means that NBA teams could have the most confidence in conclusions related to threes but should perhaps be careful when making broad conclusions about midrange shots based on the Kriging technique.



[1] Dick Brus. "Spatial sampling with R". In: CRC Press, 2023. Chap. 21 Introduction to Kriging. URL: https://dickbrus.github.io/SpatialSamplingwithR/Introkriging.html (visited on 04/24/2026).

[2] Mirko Luigi Carlesso et al. "Scoring probability maps in the basketball court with indicator kriging estimation". In: *Computational Statistics* (2024), pp. 1–21.

[3] Nicolas Christou. *Ordinary kriging in terms of the covariance function*. URL: http://www.stat.ucla.edu/~nchristo/statistics_c173_c273/c173c273_ordinary_kriging_using_covariance.pdf (visited on 04/25/2026).

[4] esri. *Semivariogram and covariance functions*. URL: https://pro.arcgis.com/en/pro-app/latest/help/analysis/geostatistical-analyst/semivariogram-and-covariance-functions.htm.

[5] esri. *Understanding ordinary kriging*. URL: https://pro.arcgis.com/en/pro-app/latest/help/analysis/geostatistical-analyst/understanding-ordinary-kriging.htm.

[6] GIS Geography. *Semi-Variogram: Nugget, Range and Sill*. URL: https://gisgeography.com/semi-variogram-nugget-range-sill/.

[7] GeostatsGuy Lectures. *Simple Kriging Lecture*. Mar. 26, 2025. URL: https://www.youtube.com/watch?v=ZRyiBKvqjJM.

[8] Kenan Li. *Kriging Interpolation in R*. Mar. 8, 2023. URL: https://kenan-li.github.io/blog/2023/Kriging/ (visited on 04/24/2026).

[9] Andrew E Long. *Derivation of the Kriging Equations*. Mar. 8, 1999. URL: https://www.nku.edu/~longa/modules/geostats/lec/latex2html/node1.html#SECTION00010000000000000000 (visited on 04/25/2026).

[10] Andrew E Long. *Variogram Modelling*. Mar. 8, 1999. URL: https://www.nku.edu/~longa/modules/geostats/lec/latex2html/node2.html (visited on 04/25/2026).

[11] Paula Moraga. "Spatial Statistics for Data Science: Theory and Practice with R". In: Chapman and Hall/CRC Data Science Series, 2023. Chap. 14 Kriging. URL: https://www.paulamoraga.com/book-spatial/kriging.html#kriging (visited on 04/23/2026).

# NBA Example Code Appendix

## Court Drawing:

```r
# Constants
rim_x <- 5.25
three_pt_r <- 23.75
corner_x <- rim_x + sqrt(three_pt_r^2 - 22^2)
paint_x <- 19
paint_y <- 8
bin_size <- 2
min_attempts <- 10

# Function to draft the lines of a basketball halfcourt
draw_court <- function() {

  # Define the angle of the three point line
  angle_top <- atan2(22, corner_x - rim_x)
  angle_bot <- atan2(-22, corner_x - rim_x)

  # Build the three point line using arc points
  arc_points <- data.frame(
    x = rim_x + three_pt_r * cos(seq(angle_top, angle_bot, length.out = 1000)),
    y = three_pt_r * sin(seq(angle_top, angle_bot, length.out = 1000))
  )
```

```r
  # Make a list of the output elements
  list(
    geom_path(data = arc_points, aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = c(0, corner_x), y = c(22, 22)),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = c(0, corner_x), y = c(-22, -22)),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = c(0, 19, 19, 0, 0), y = c(8, 8, -8, -8, 8)),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = 19 + 6 * cos(seq(-pi/2, pi/2, length.out = 500)),
                                y = 6 * sin(seq(-pi/2, pi/2, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = 19 + 6 * cos(seq(pi/2, 3*pi/2, length.out = 500)),
                                y = 6 * sin(seq(pi/2, 3*pi/2, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6,
              linetype = "dashed"),
    geom_path(data = data.frame(x = rim_x + 4 * cos(seq(-pi/2, pi/2, length.out = 500)),
                                y = 4 * sin(seq(-pi/2, pi/2, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = rim_x + 0.75 * cos(seq(0, 2*pi, length.out = 500)),
                                y = 0.75 * sin(seq(0, 2*pi, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "darkorange2", linewidth = 0.8)
  )
}
```

# Data Cleaning

```r
shots <- pbp %>%
  filter(shooting_play == TRUE,
         !is.na(coordinate_x),
         !is.na(coordinate_y),
         !grepl("Free Throw", type_text),
         season_type == 2) %>%
  left_join(player_names, by = c("athlete_id_1" = "athlete_id")) %>%
  mutate(
    team_name = ifelse(team_id == home_team_id,
                       paste(home_team_name, home_team_mascot),
                       paste(away_team_name, away_team_mascot)),
    x = ifelse(coordinate_x > 0, -(coordinate_x - 46.75), coordinate_x + 46.75),
    y = ifelse(coordinate_x > 0, -coordinate_y, coordinate_y)
  ) %>%
  filter(x <= 45) %>%
  select(athlete_display_name, team_name, score_value, x, y)
```

# Prediction Grid Helper Function

```r
make_pred_grid <- function() {
  pred_grid <- expand.grid(
    x = seq(0.5, 45.5, by = 1),
    y = seq(-24.5, 25.5, by = 1)
  ) %>%
    as.data.frame() %>%
    mutate(dist_from_rim = sqrt((x - rim_x)^2 + y^2)) %>%
    filter(dist_from_rim <= three_pt_r + 5) %>%
    select(x, y) %>%
    as.data.frame()

  coordinates(pred_grid) <- ~x+y
  gridded(pred_grid) <- TRUE
  pred_grid
}
```

# Bin-Making Helper Function

```r
make_bins <- function(team, bin_size = 2, min_attempts = 10) {
  team_shots %>%
    filter(team_name == team) %>%
    mutate(
      x_bin = floor(x / bin_size) * bin_size + bin_size / 2,
      y_bin = floor(y / bin_size) * bin_size + bin_size / 2
    ) %>%
    group_by(x_bin, y_bin) %>%
    summarise(
      attempts = n(),
      makes = sum(score_value > 0),
      fg_pct = makes / attempts,
      .groups = "drop"
    ) %>%
    filter(attempts >= min_attempts) %>%
    mutate(
      dist_from_rim = sqrt((x_bin - rim_x)^2 + y_bin^2),
      is_three = dist_from_rim >= three_pt_r | (abs(y_bin) >= 22 & x_bin <= corner_x),
      in_paint = x_bin <= paint_x & abs(y_bin) <= paint_y,
      zone = case_when(
        is_three ~ "Three",
        in_paint ~ "Paint",
        TRUE ~ "Midrange"
      )
    ) %>%
    as.data.frame()
}
```

# plot_attempts Helper Function

```r
team_shots <- shots %>% select(team_name, score_value, x, y)

plot_attempts <- function(team) {
  pred_grid <- make_pred_grid()
  bins <- make_bins(team)
  attempt_bins <- bins %>% select(x_bin, y_bin, attempts) %>% as.data.frame()
  coordinates(attempt_bins) <- ~x_bin+y_bin

  krig_attempts <- suppressWarnings(
    autoKrige(attempts ~ 1, input_data = attempt_bins, new_data = pred_grid)
  )

  krig_attempts_df <- as.data.frame(krig_attempts$krige_output)
  names(krig_attempts_df) <- c("x", "y", "attempts_pred", "attempts_var", "attempts_stdev")
  krig_attempts_df <- krig_attempts_df %>% mutate(attempts_pred = pmax(0, attempts_pred))

  ggplot(krig_attempts_df, aes(x = x, y = y, fill = attempts_pred)) +
    geom_raster(interpolate = TRUE) +
    draw_court() +
    scale_fill_viridis_c() +
    coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
    labs(title = paste(team, "- Shot Attempts"),
         x = "Distance from Basket (ft)",
         y = "Court Width (ft)",
         fill = "Attempts") +
    theme_minimal(base_size = 7) +
    theme(panel.grid = element_blank(),
          plot.margin = margin(t = 2, r = 0, b = 0, l = -3))
```

# plot_fg_pct Helper Function

```r
plot_fg_pct <- function(team) {
  pred_grid <- make_pred_grid()
  bins <- make_bins(team)
  fg_bins <- bins %>% select(x_bin, y_bin, fg_pct) %>% as.data.frame()
  coordinates(fg_bins) <- ~x_bin+y_bin

  krig_fg_auto <- suppressMessages(
    suppressWarnings(autoKrige(fg_pct ~ 1, input_data = fg_bins, new_data = pred_grid))
  )

  krig_fg_df <- as.data.frame(krig_fg_auto$krige_output)
  names(krig_fg_df) <- c("x", "y", "fg_pred", "fg_var", "fg_stdev")

  return(list(
    plot = ggplot(krig_fg_df, aes(x = x, y = y, fill = fg_pred)) +
      geom_raster(interpolate = TRUE) +
      draw_court() +
      scale_fill_viridis_c(labels = scales::percent,
                           limits = c(0.20, 0.60),
                           oob = scales::squish) +
      coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
      labs(title = paste(team, "- FG%"),
           x = "Distance from Basket (ft)",
           y = "Court Width (ft)",
           fill = "FG%") +
      theme_minimal(base_size = 7) +
      theme(panel.grid = element_blank(),
            plot.margin = margin(t = 2, r = 0, b = 0, l = -3)),
```

# plot_pps Helper Function

```r
# Define a function to perform Kriging on the points per shot data
plot_pps <- function(fg_data, team = "") {

  # Define the number of points each shot is worth
  fg_data %>%
    mutate(dist_from_rim = sqrt((x - rim_x)^2 + y^2),
      is_three = dist_from_rim >= three_pt_r | (abs(y) >= 22 & x <= corner_x),
      point_value = ifelse(is_three, 3, 2),pps = fg_pred * point_value) %>%

    # Plot the data using helper functions
    ggplot(aes(x = x, y = y, fill = pps)) +
    geom_raster(interpolate = TRUE) +
    draw_court() +
    scale_fill_viridis_c(limits = c(0.5, 2.0), oob = scales::squish) +
    coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
    labs(title = paste(team, "- Points per Shot"),
         x = "Distance from Basket (ft)",
         y = "Court Width (ft)",
         fill = "PPS") +
    theme_minimal(base_size = 7) +
    theme(panel.grid = element_blank(), plot.margin = margin(t = 2, r = 0, b = 0, l = -3))
}
```

# NBA Example Code Appendix

## Court Drawing:

```r
# Constants
rim_x <- 5.25
three_pt_r <- 23.75
corner_x <- rim_x + sqrt(three_pt_r^2 - 22^2)
paint_x <- 19
paint_y <- 8
bin_size <- 2
min_attempts <- 10

# Function to draft the lines of a basketball halfcourt
draw_court <- function() {

  # Define the angle of the three point line
  angle_top <- atan2(22, corner_x - rim_x)
  angle_bot <- atan2(-22, corner_x - rim_x)

  # Build the three point line using arc points
  arc_points <- data.frame(
    x = rim_x + three_pt_r * cos(seq(angle_top, angle_bot, length.out = 1000)),
    y = three_pt_r * sin(seq(angle_top, angle_bot, length.out = 1000))
  )
```

```r
  # Make a list of the output elements
  list(
    geom_path(data = arc_points, aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = c(0, corner_x), y = c(22, 22)),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = c(0, corner_x), y = c(-22, -22)),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = c(0, 19, 19, 0, 0), y = c(8, 8, -8, -8, 8)),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = 19 + 6 * cos(seq(-pi/2, pi/2, length.out = 500)),
                                y = 6 * sin(seq(-pi/2, pi/2, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = 19 + 6 * cos(seq(pi/2, 3*pi/2, length.out = 500)),
                                y = 6 * sin(seq(pi/2, 3*pi/2, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6,
              linetype = "dashed"),
    geom_path(data = data.frame(x = rim_x + 4 * cos(seq(-pi/2, pi/2, length.out = 500)),
                                y = 4 * sin(seq(-pi/2, pi/2, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "black", linewidth = 0.6),
    geom_path(data = data.frame(x = rim_x + 0.75 * cos(seq(0, 2*pi, length.out = 500)),
                                y = 0.75 * sin(seq(0, 2*pi, length.out = 500))),
              aes(x = x, y = y), inherit.aes = FALSE, color = "darkorange2", linewidth = 0.8)
  )
}
```

# Data Cleaning

```r
shots <- pbp %>%
  filter(shooting_play == TRUE,
         !is.na(coordinate_x),
         !is.na(coordinate_y),
         !grepl("Free Throw", type_text),
         season_type == 2) %>%
  left_join(player_names, by = c("athlete_id_1" = "athlete_id")) %>%
  mutate(
    team_name = ifelse(team_id == home_team_id,
                       paste(home_team_name, home_team_mascot),
                       paste(away_team_name, away_team_mascot)),
    x = ifelse(coordinate_x > 0, -(coordinate_x - 46.75), coordinate_x + 46.75),
    y = ifelse(coordinate_x > 0, -coordinate_y, coordinate_y)
  ) %>%
  filter(x <= 45) %>%
  select(athlete_display_name, team_name, score_value, x, y)
```

# Prediction Grid Helper Function

```r
make_pred_grid <- function() {
  pred_grid <- expand.grid(
    x = seq(0.5, 45.5, by = 1),
    y = seq(-24.5, 25.5, by = 1)
  ) %>%
    as.data.frame() %>%
    mutate(dist_from_rim = sqrt((x - rim_x)^2 + y^2)) %>%
    filter(dist_from_rim <= three_pt_r + 5) %>%
    select(x, y) %>%
    as.data.frame()

  coordinates(pred_grid) <- ~x+y
  gridded(pred_grid) <- TRUE
  pred_grid
}
```

# Bin-Making Helper Function

```r
make_bins <- function(team, bin_size = 2, min_attempts = 10) {
  team_shots %>%
    filter(team_name == team) %>%
    mutate(
      x_bin = floor(x / bin_size) * bin_size + bin_size / 2,
      y_bin = floor(y / bin_size) * bin_size + bin_size / 2
    ) %>%
    group_by(x_bin, y_bin) %>%
    summarise(
      attempts = n(),
      makes = sum(score_value > 0),
      fg_pct = makes / attempts,
      .groups = "drop"
    ) %>%
    filter(attempts >= min_attempts) %>%
    mutate(
      dist_from_rim = sqrt((x_bin - rim_x)^2 + y_bin^2),
      is_three = dist_from_rim >= three_pt_r | (abs(y_bin) >= 22 & x_bin <= corner_x),
      in_paint = x_bin <= paint_x & abs(y_bin) <= paint_y,
      zone = case_when(
        is_three ~ "Three",
        in_paint ~ "Paint",
        TRUE ~ "Midrange"
      )
    ) %>%
    as.data.frame()
}
```

# plot_attempts Helper Function

```r
team_shots <- shots %>% select(team_name, score_value, x, y)

plot_attempts <- function(team) {
  pred_grid <- make_pred_grid()
  bins <- make_bins(team)
  attempt_bins <- bins %>% select(x_bin, y_bin, attempts) %>% as.data.frame()
  coordinates(attempt_bins) <- ~x_bin+y_bin

  krig_attempts <- suppressWarnings(
    autoKrige(attempts ~ 1, input_data = attempt_bins, new_data = pred_grid)
  )

  krig_attempts_df <- as.data.frame(krig_attempts$krige_output)
  names(krig_attempts_df) <- c("x", "y", "attempts_pred", "attempts_var", "attempts_stdev")
  krig_attempts_df <- krig_attempts_df %>% mutate(attempts_pred = pmax(0, attempts_pred))

  ggplot(krig_attempts_df, aes(x = x, y = y, fill = attempts_pred)) +
    geom_raster(interpolate = TRUE) +
    draw_court() +
    scale_fill_viridis_c() +
    coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
    labs(title = paste(team, "- Shot Attempts"),
         x = "Distance from Basket (ft)",
         y = "Court Width (ft)",
         fill = "Attempts") +
    theme_minimal(base_size = 7) +
    theme(panel.grid = element_blank(),
          plot.margin = margin(t = 2, r = 0, b = 0, l = -3))
```

# plot_fg_pct Helper Function

```r
plot_fg_pct <- function(team) {
  pred_grid <- make_pred_grid()
  bins <- make_bins(team)
  fg_bins <- bins %>% select(x_bin, y_bin, fg_pct) %>% as.data.frame()
  coordinates(fg_bins) <- ~x_bin+y_bin

  krig_fg_auto <- suppressMessages(
    suppressWarnings(autoKrige(fg_pct ~ 1, input_data = fg_bins, new_data = pred_grid))
  )

  krig_fg_df <- as.data.frame(krig_fg_auto$krige_output)
  names(krig_fg_df) <- c("x", "y", "fg_pred", "fg_var", "fg_stdev")

  return(list(
    plot = ggplot(krig_fg_df, aes(x = x, y = y, fill = fg_pred)) +
      geom_raster(interpolate = TRUE) +
      draw_court() +
      scale_fill_viridis_c(labels = scales::percent,
                           limits = c(0.20, 0.60),
                           oob = scales::squish) +
      coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
      labs(title = paste(team, "- FG%"),
           x = "Distance from Basket (ft)",
           y = "Court Width (ft)",
           fill = "FG%") +
      theme_minimal(base_size = 7) +
      theme(panel.grid = element_blank(),
            plot.margin = margin(t = 2, r = 0, b = 0, l = -3)),
```

# plot_pps Helper Function

```r
# Define a function to perform Kriging on the points per shot data
plot_pps <- function(fg_data, team = "") {

  # Define the number of points each shot is worth
  fg_data %>%
    mutate(dist_from_rim = sqrt((x - rim_x)^2 + y^2),
      is_three = dist_from_rim >= three_pt_r | (abs(y) >= 22 & x <= corner_x),
      point_value = ifelse(is_three, 3, 2),pps = fg_pred * point_value) %>%

    # Plot the data using helper functions
    ggplot(aes(x = x, y = y, fill = pps)) +
    geom_raster(interpolate = TRUE) +
    draw_court() +
    scale_fill_viridis_c(limits = c(0.5, 2.0), oob = scales::squish) +
    coord_fixed(xlim = c(0, 35), ylim = c(-25, 25)) +
    labs(title = paste(team, "- Points per Shot"),
         x = "Distance from Basket (ft)",
         y = "Court Width (ft)",
         fill = "PPS") +
    theme_minimal(base_size = 7) +
    theme(panel.grid = element_blank(), plot.margin = margin(t = 2, r = 0, b = 0, l = -3))
}
```

[1] Dick Brus. "Spatial sampling with R". In: CRC Press, 2023. Chap. 21 Introduction to Kriging. URL: https://dickbrus.github.io/SpatialSamplingwithR/Introkriging.html (visited on 04/24/2026).

[2] Mirko Luigi Carlesso et al. "Scoring probability maps in the basketball court with indicator kriging estimation". In: *Computational Statistics* (2024), pp. 1–21.

[3] Nicolas Christou. *Ordinary kriging in terms of the covariance function*. URL: http://www.stat.ucla.edu/~nchristo/statistics_c173_c273/c173c273_ordinary_kriging_using_covariance.pdf (visited on 04/25/2026).

[4] esri. *Semivariogram and covariance functions*. URL: https://pro.arcgis.com/en/pro-app/latest/help/analysis/geostatistical-analyst/semivariogram-and-covariance-functions.htm.

[5] esri. *Understanding ordinary kriging*. URL: https://pro.arcgis.com/en/pro-app/latest/help/analysis/geostatistical-analyst/understanding-ordinary-kriging.htm.

[6] GIS Geography. *Semi-Variogram: Nugget, Range and Sill*. URL: https://gisgeography.com/semi-variogram-nugget-range-sill/.

[7] GeostatsGuy Lectures. *Simple Kriging Lecture*. Mar. 26, 2025. URL: https://www.youtube.com/watch?v=ZRyiBKvqjJM.

[8] Kenan Li. *Kriging Interpolation in R*. Mar. 8, 2023. URL: https://kenan-li.github.io/blog/2023/Kriging/ (visited on 04/24/2026).

[9] Andrew E Long. *Derivation of the Kriging Equations*. Mar. 8, 1999. URL: https://www.nku.edu/~longa/modules/geostats/lec/latex2html/node1.html#SECTION00010000000000000000 (visited on 04/25/2026).

[10] Andrew E Long. *Variogram Modelling*. Mar. 8, 1999. URL: https://www.nku.edu/~longa/modules/geostats/lec/latex2html/node2.html (visited on 04/25/2026).

[11] Paula Moraga. "Spatial Statistics for Data Science: Theory and Practice with R". In: Chapman and Hall/CRC Data Science Series, 2023. Chap. 14 Kriging. URL: https://www.paulamoraga.com/book-spatial/kriging.html#kriging (visited on 04/23/2026).